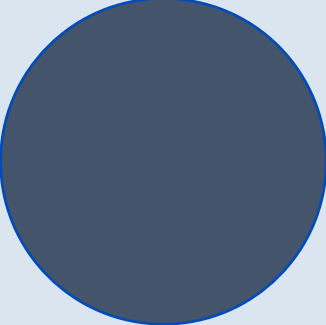
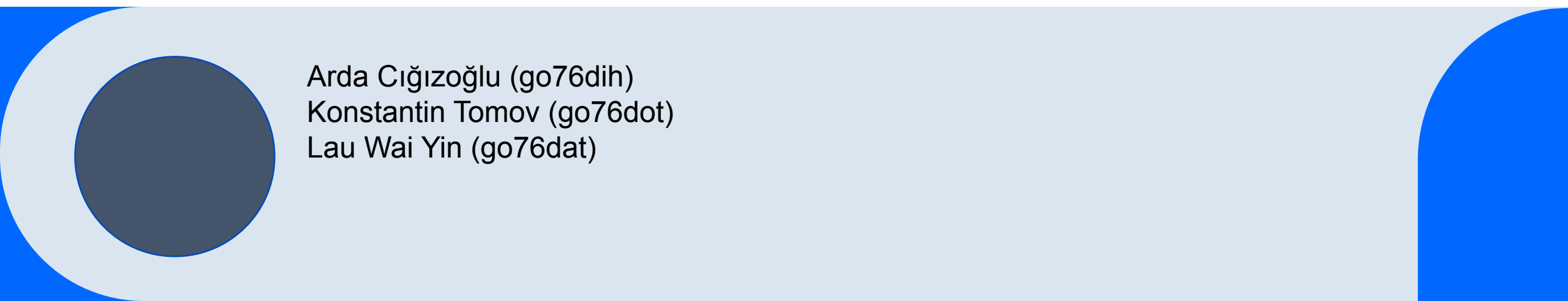




GRA

ABSCHLUSSPROJEKT

Memory Cache



Arda Cıgızoğlu (go76dih)
Konstantin Tomov (go76dot)
Lau Wai Yin (go76dat)

Contents

Introduction

Implementation

Testing

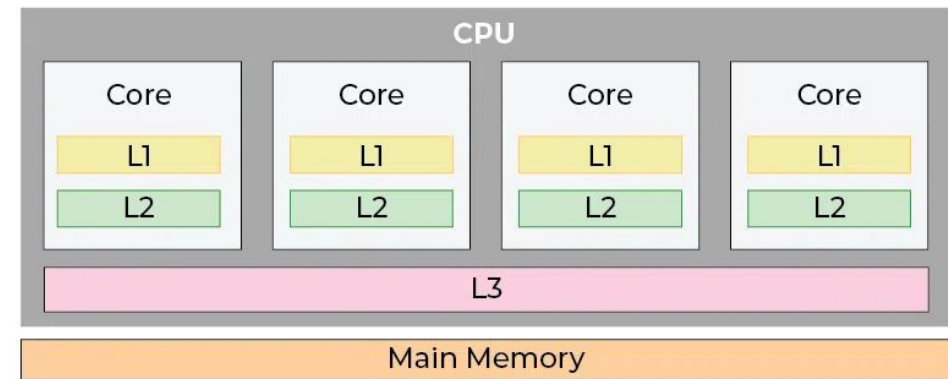
Summary

What is a cache?

Temporary storage

- **A small, quick-access memory** that stores information you use often, so your device doesn't have to do all the work to get it again.
- Fetching from main memory is slow, but from cache is fast
- Spatial and temporal locality:
- memories used more likely to be used again
- memories closed together more likely to be used

Possible Implementation



CPU Cache

Source:
Geeksforgeeks



Some facts about the cache

Memory Type	Speed (Latency)	Relative Speed	Typical Size
L1 Cache	~0.5 ns	Fastest ($\times 1$)	16–64 KB
L2 Cache	~3–10 ns	Slower ($\times 6$ –20)	256 KB–1 MB
L3 Cache	~10–30 ns	Slower ($\times 20$ –60)	4–64 MB
Main Memory	~100 ns	Much slower ($\times 100$ –200)	GBs (e.g., 8–64 GB)

Source: Computer Organization and Design" by Patterson & Hennessy

How does a cache store memory

A **cache line** is the smallest unit of data storage in the cache. Each cache level is composed of many such cache lines. It typically contains:

The actual data (a block from main memory)

A **tag** (to identify which memory block it is storing)

A **valid bit** (to mark if the data is usable)

Cache hit and miss

If memory found in cache (Hit)

Reading memory: The CPU just fetches the required memory (fast)

Writing memory: Two policies

1. **Write-through:** Write to **cache and main memory**.
2. **Write-back:** Write **only to cache** now, and **update main memory later** (on eviction).

If not in cache (Miss)

Reading memory: searches the caches below or the main memory, then load the memory into the cache for faster access next time

Writing memory: (Two policies)

1. **Write-allocate** (a.k.a. fetch-on-write):
Load the block into the cache, then write to it. Works well with **write-back** caches.
2. **No-write-allocate:**
Write **directly to memory**, do **not** load into cache. Often used with **write-through** caches.

Caching policy

The **mapping strategy** defines how main memory blocks are stored in the cache:

Set-Associative: The cache level is divided into sets holding a number of cache lines. Memory blocks map to precisely one set and are stored into one of its cache lines, based on the mapping strategy used.

Direct-Mapped: Each memory block maps to precisely one cache line. Often leads to collisions (two blocks competing for the same line). This is a special case of 1 set-associative.

Fully Associative: Any memory block can go into any cache line. This minimizes collisions, but is more complex.

Cache conflicts (Replacement policy)

LRU: Removes least recently used information.

Pros: Simple with good general performance

Cons: Poor performance when looping over large sets.

LFU: Removes least frequently used information.

Pros: Optimal at using information that has hot(very frequently used) values.

Cons: Not adaptive, as old hot values will remain in the cache.

FIFO: Removes oldest inserted element.

Pros: No need for extra bookkeeping and simple.

Cons: Ignores usage patterns and can evict frequently accessed elements.

Random Replacement: Removes a random element.

Pros: Avoids worst cases of the LFU and LRU strategies.

Cons: Unpredictable as it can remove valuable and non-valuable data with the same probability.

MRU: Removes most recently accessed item.

Pros: Works well with backtracking algorithms.

Cons: Does opposite of what common usage patterns suggest.

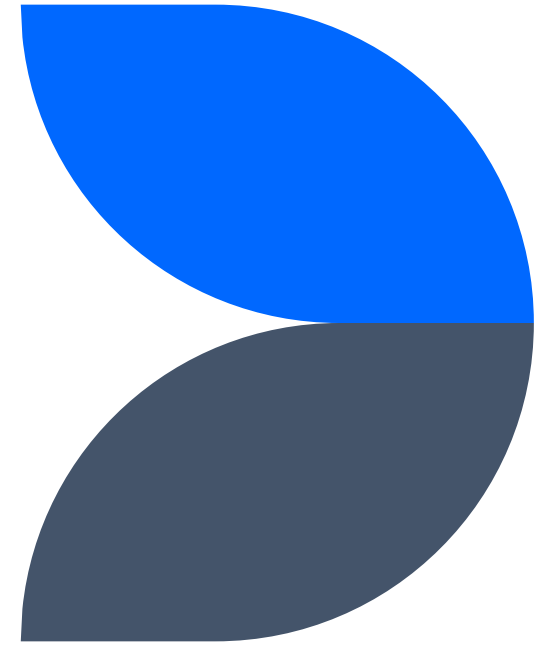
Belady's algorithm: Removes information which will not be needed for the longest time.

Pros: Most optimal strategy in theory.

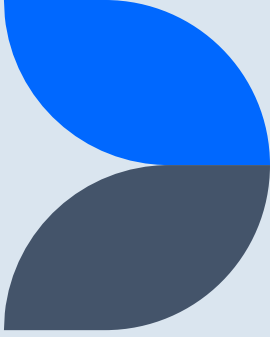
Cons: Unusable in practice, as it requires knowledge of future information

Our Implementation

Memory cache for a single
processor with Systemc



Cache Options



- Selection of number of cache lines, cache sizes, levels, and latency
- Caching strategy: Direct-mapped, set-associative and fully-associative
- Replacement Strategy: LRU, LFU, FIFO, MRU, RANDOM
- Generate trace file option

Caching

Read

Hit: add cache line to all levels below.

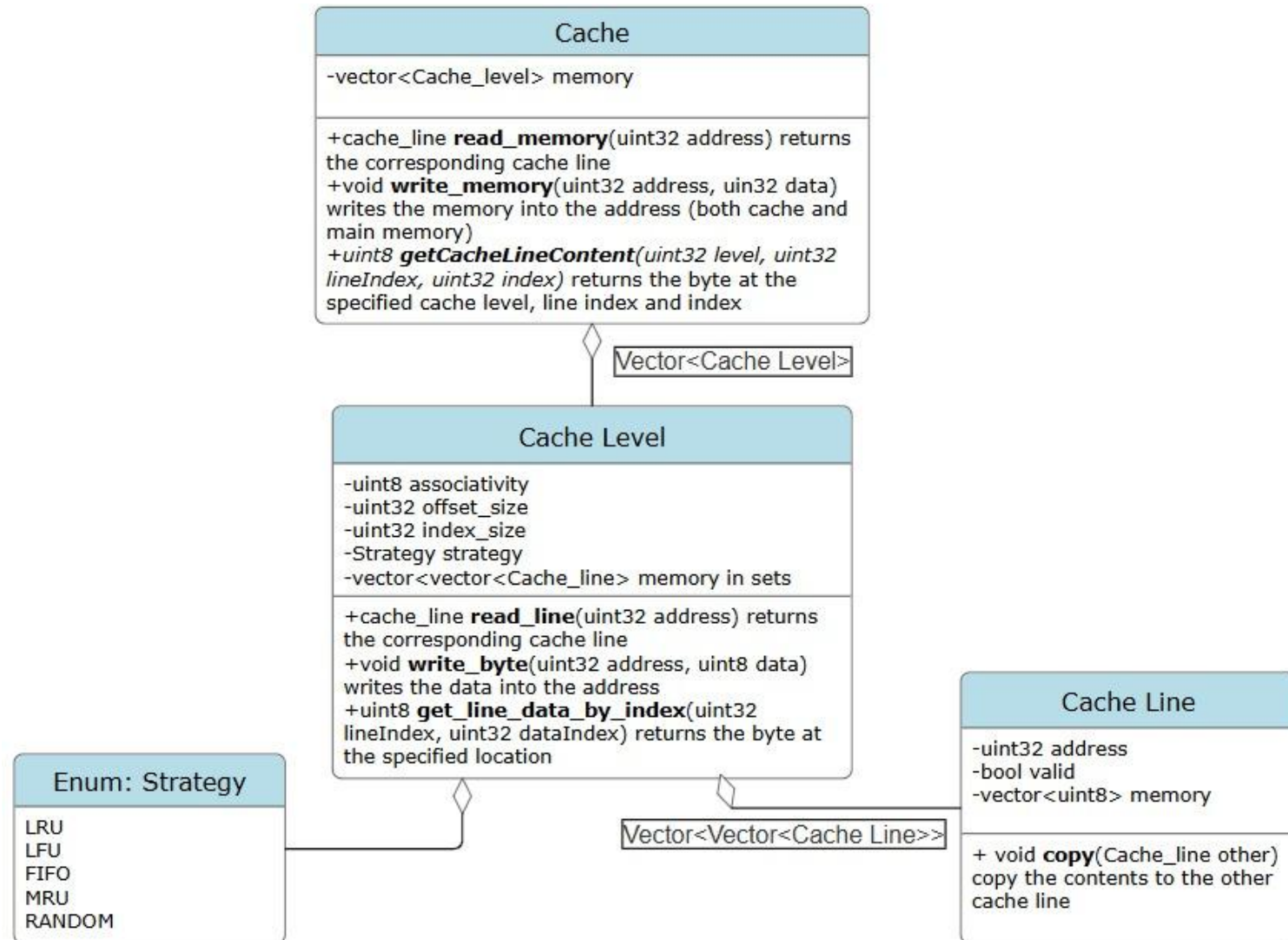
Miss: retrieve from the main memory, create a new cache line, and add it to all levels. Then append the byte corresponding to the current address to our result data.

Write

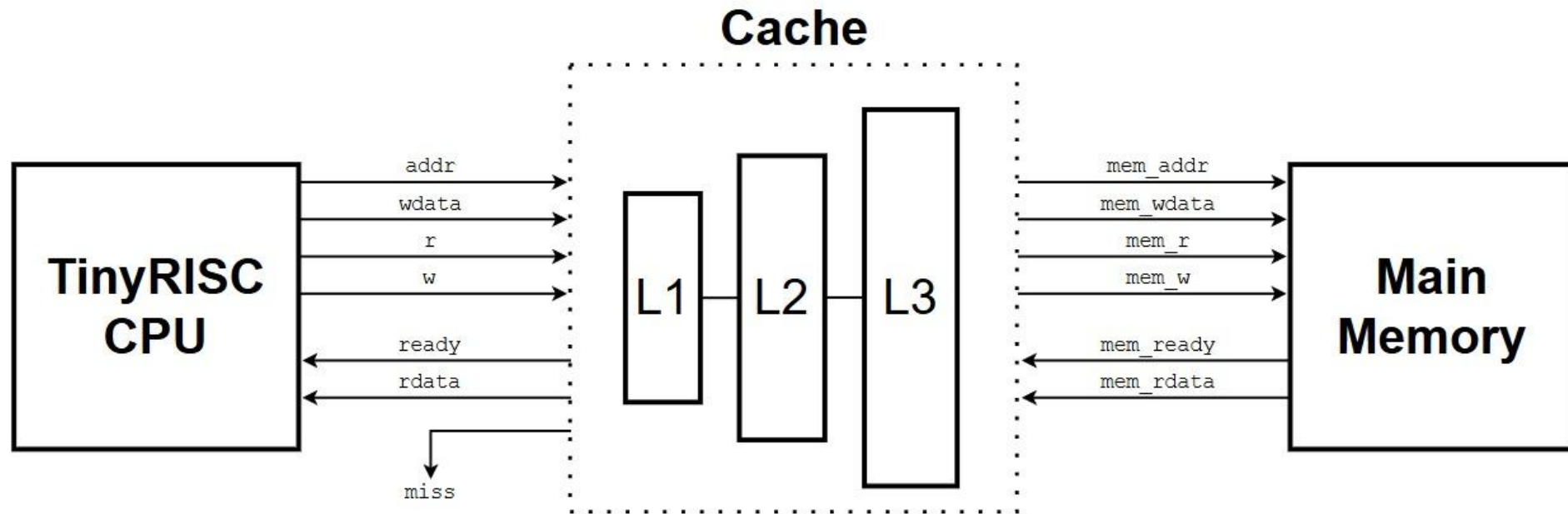
Hit: write the new byte to this line, update all levels above and then update the main memory

Miss: update main memory and add it to all cache lines.

Structure: Simplified for Viewing

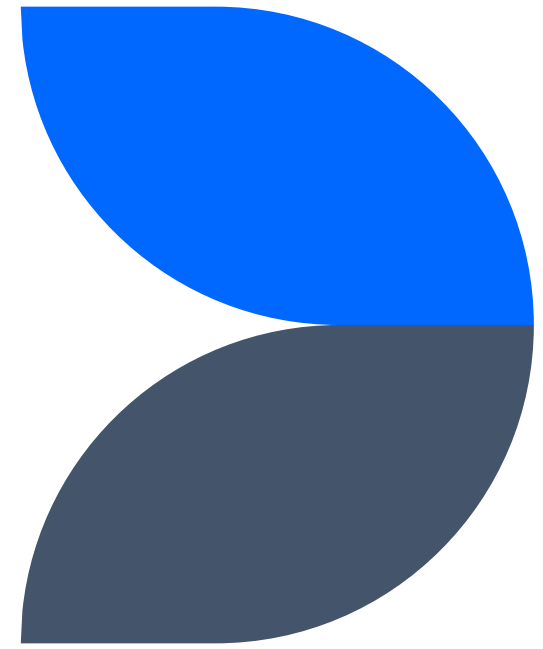


RISC CPU Integration



Testing

On generated memory accesses



Random Data - 12 Bit Addresses

4-way Associative

L1 level with 128 lines and 32 bits per line			
Strategy	Cycles	Misses	Hits
LRU	5669317	337	99663
LFU	5612341	267	99733
FIFO	5719009	398	99602
MRU	5961573	696	99304
RANDOM	5732857	415	99585

Random Data - 12 Bit Addresses Directly Mapped

L1 level with 128 lines and 32 bits per line			
Strategy	Cycles	Misses	Hits
LRU	5612341	267	99733
LFU	5612341	267	99733
FIFO	5612341	267	99733
MRU	5612341	267	99733
RANDOM	5612341	267	99733

Random Data - 12 Bit Addresses

Fully-Associative

L1 level with 128 lines and 32 bits per line			
Strategy	Cycles	Misses	Hits
LRU	5786529	481	99519
LFU	5615581	271	99729
FIFO	6122001	886	99114
MRU	6249697	1050	98950
RANDOM	6127725	898	99102

Matrix Multiplication Data - 50x20x30

4-way Associative

L1 level with 128 lines and 32 bits per line			
Strategy	Cycles	Misses	Hits
LRU	3898093	441	119559
LFU	10469609	8534	111466
FIFO	4072673	656	119344
MRU	10092841	8070	111930
RANDOM	4123829	719	119281

Matrix Multiplication Data - 50x20x30 Directly Mapped

L1 level with 128 lines and 32 bits per line			
Strategy	Cycles	Misses	Hits
LRU	5355633	2236	117764
LFU	~	~	~
FIFO	~	~	~
MRU	~	~	~
RANDOM	~	~	~

Matrix Multiplication Data - 50x20x30

Fully-Associative

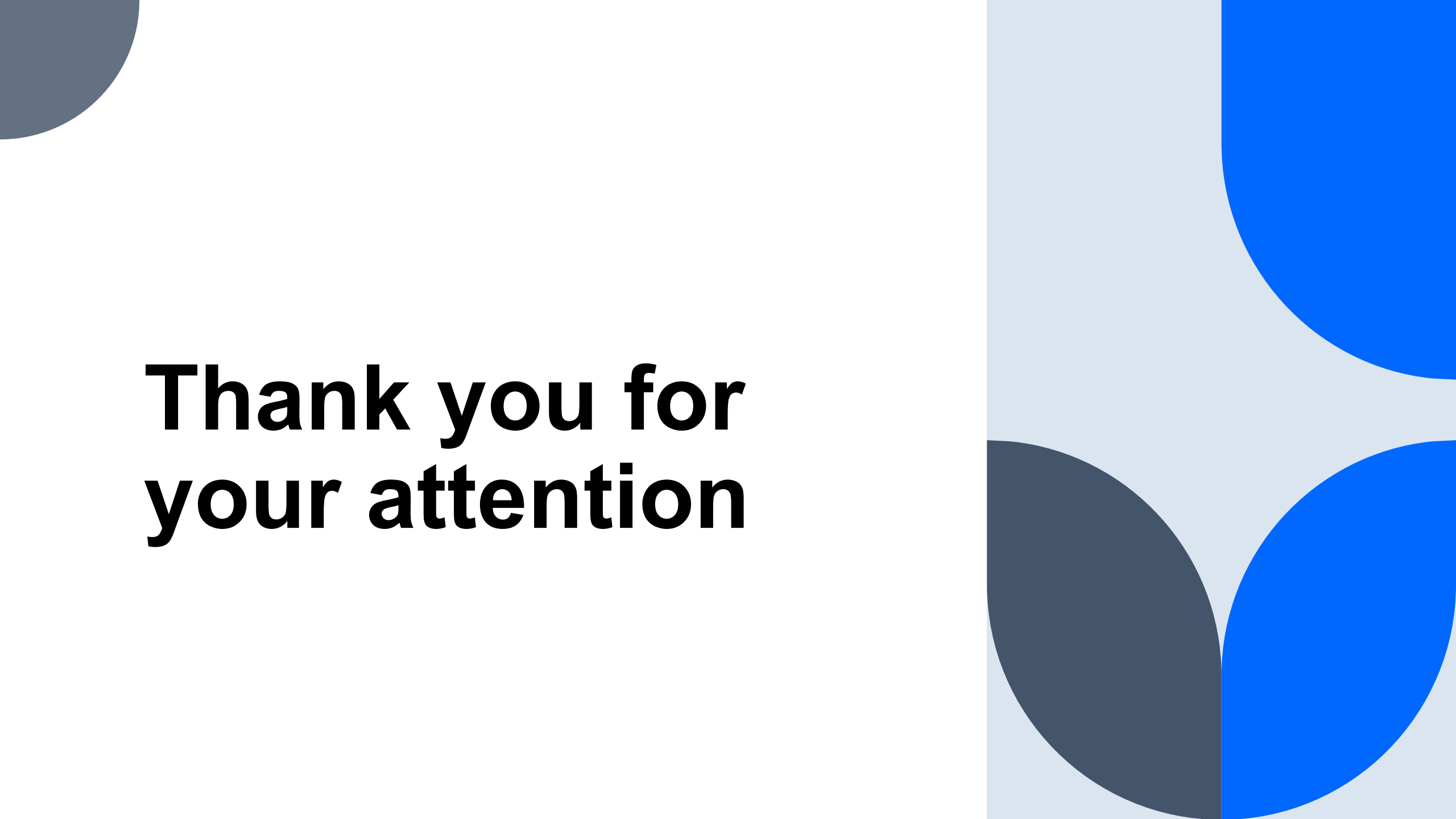
L1 level with 128 lines and 32 bits per line			
Strategy	Cycles	Misses	Hits
LRU	3855057	388	119612
LFU	44041749	49879	70121
FIFO	4159557	763	119237
MRU	32548701	35725	84275
RANDOM	4152249	754	119246

Matrix Multiplication Data - 50x20x30 Supplements

Strategy	Cycles	Misses	Hits
Bad traversal - L1 level with 128 lines and 32 bits per line			
LRU - Fully Associative	7926425	5402	114598
LFU - Fully Associative	52328209	60084	59916
Bad traversal - L1 level with 128 lines, L2 level with 256 lines, and 32 bits per line			
MRU	3855057	388	119612

Summary

We built a memory cache in systemC, supporting Direct-mapped, set-associative and fully-associative caching strategies. Moreover, it supports replacement strategy: LRU, LFU, FIFO, MRU, RANDOM



**Thank you for
your attention**